

## Performance Testing

|                      |                                 |
|----------------------|---------------------------------|
| <b>Date</b>          | 5 <sup>th</sup> July 2026       |
| <b>Team ID</b>       | RAHMA HUSSEIN JUMA              |
| <b>Project Name</b>  | CREDIT CARD APPROVAL PREDICTION |
| <b>Maximum Marks</b> | 5 Marks                         |

### Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

#### Step 1: Testing Overview

| Field                      | Details                                                                                            |
|----------------------------|----------------------------------------------------------------------------------------------------|
| <b>Testing Tool Used</b>   | Custom Python script (requests + threading for load generation, ps/psutil for resource monitoring) |
| <b>Type of Testing</b>     | Load Testing                                                                                       |
| <b>Target Module / API</b> | /predict endpoint (Flask backend)                                                                  |
| <b>Test Environment</b>    | Local (Flask development server)                                                                   |
| <b>Test Date</b>           | July 6, 2026                                                                                       |

#### Step 2: Test Scenarios

| S.No | Test Scenario / Description                                          | No. of Virtual Users | Duration (sec) | Expected Outcome                                                              |
|------|----------------------------------------------------------------------|----------------------|----------------|-------------------------------------------------------------------------------|
| 1    | Light load — a few applicants submitting checks around the same time | 5                    | 0.12           | All requests succeed with correct predictions and fast response times         |
| 2    | Moderate concurrent load — typical busy-period usage                 | 20                   | 0.52           | All requests succeed with no errors, response times stay well under 2 seconds |

|   |                                                                             |    |      |                                                                                          |
|---|-----------------------------------------------------------------------------|----|------|------------------------------------------------------------------------------------------|
| 3 | Heavy concurrent load — stress test beyond expected normal usage            | 50 | 1.62 | System remains stable with 0% errors, even if response times increase under heavier load |
| 4 | Home page access load — multiple users loading the form page simultaneously | 20 | 0.19 | Form page loads quickly and reliably for all simultaneous visitors                       |



### Step 3: Performance Test Results

| S.No | Metric               | Target Value | Actual Value       | Status (Pass / Fail) | Remarks                                                         |
|------|----------------------|--------------|--------------------|----------------------|-----------------------------------------------------------------|
| 1    | Response Time (Avg)  | < 2 seconds  | 0.255 sec          | Pass                 | Well within target                                              |
| 2    | Response Time (Max)  | < 5 seconds  | 0.390 sec          | Pass                 |                                                                 |
| 3    | Throughput (Req/sec) |              | 195.8 req/sec      | Pass                 | Measured on local dev server, single process                    |
| 4    | Error Rate           | < 1%         | 0.00%              | Pass                 | 100/100 requests succeeded                                      |
| 5    | CPU Utilization      | < 80%        | 40.3%(peak)        | Pass                 | Measured on Flask's single process during the 100-request burst |
| 6    | Memory Utilization   | < 80%        | ~161 MB RSS (peak) | Pass                 | Stable, no memory growth pattern suggesting a leak              |

### Step 4: Observations & Analysis

#### Key Findings:

The app handled 100 requests from 20 users hitting it at the same time in about half a second, with zero errors. Response times stayed fast the whole way through — nothing took longer than 0.4 seconds. CPU usage went up to about 40% at peak, so there was still plenty of room to spare.

#### Bottlenecks Identified:

Nothing showed up at this scale. If there's a weak point, it's not the model — it's that we're running Flask's built-in dev server, which is single-threaded and was never meant to handle real production traffic. That would start to matter if usage got a lot heavier than what we tested here.

#### Optimization Steps Taken:

Didn't need to optimize anything for this test — it handled the load fine as-is. If this ever went to production, the obvious next step would be switching to a proper WSGI server like Gunicorn (so it can

actually run multiple workers), or hosting the model on IBM Watson ML like originally planned, so it's not all riding on one local server.

### Step 5: Screenshots / Evidence

```
=== Load Test Results ===
```

```
Total requests sent: 100
```

```
Successful responses: 100
```

```
Errors: 0
```

```
Total wall time: 0.51 sec
```

```
Throughput: 195.77 req/sec
```

```
Avg response time: 0.255 sec
```

```
Max response time: 0.390 sec
```

```
Min response time: 0.073 sec
```

```
95th percentile: 0.384 sec
```

```
Error rate: 0.00%
```

```
Peak CPU utilization (Flask process): 40.3%
```

```
Peak memory usage (Flask process): 161.2 MB
```